SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO2: Design UML diagrams to represent system requirements and architecture. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: Any 4 Total Marks:40 Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I _Bheeshma (192511332) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Bheeshma L

Annexure A

ANALYTICAL PROBLEM SOLVING

(Completed Assessment — Any 4 Questions Answered)

1. Use Case Diagram Problem

A **Hospital Management System** allows patients to book appointments, doctors to update medical records, and administrators to manage hospital resources. The requirements are not clearly documented.

Tasks:
a) Identify all actors involved in the system.
b) List the primary use cases.
c) Define relationships (include, extend, generalization).
d) Construct a complete Use Case Diagram representing system functionality.

(10 Marks)

Answer

a) Actors Identified

- **Patient** — books appointments, makes payments, views own records.
- **Doctor** — updates medical records, views appointment schedule.
- **Administrator** — manages hospital resources (rooms, staff, equipment) and generates reports. ● **Payment Gateway** (secondary/external actor) — processes appointment payments.

b) Primary Use Cases

- Login / Authenticate
- Book Appointment
- Cancel Appointment
- Make Payment
- View Medical Records
- Update Medical Records
- Manage Hospital Resources
- Generate Reports

c) Relationships Defined

- **<<include>>:** "Book Appointment" includes "Login/Authenticate" and "Make Payment", since these steps must always occur for a booking to complete.
- **<<include>>:** "Cancel Appointment" includes "Login/Authenticate".
- **<<extend>>:** "Generate Reports" extends "Manage Hospital Resources" — it is an optional step performed only when the administrator requests analytics.
- **Generalization:** Doctor and Administrator can both be generalized from a common "Hospital Staff" actor, since both must log in through the same authentication use case.

d) Use Case Diagram

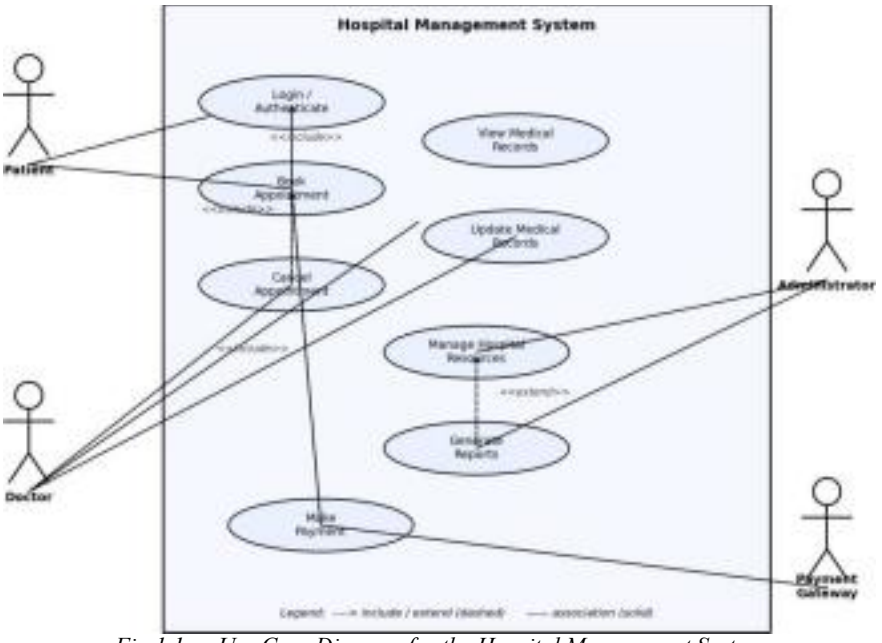

Fig 1.1 — Use Case Diagram for the Hospital Management System

2. Class Diagram Problem

A **Library Management System** contains entities such as Book, Member, Librarian, and IssueRecord. The relationships among entities are not properly defined.

Tasks:
a) Identify classes with attributes and methods.
b) Define relationships (association, aggregation, inheritance).
c) Specify multiplicity between classes.
d) Design a detailed Class Diagram for the system.

(10 Marks)

Answer

a) Classes with Attributes and Methods

- **Book** — bookId, title, author, status; methods: checkAvailability(), updateStatus().
- **Member** — memberId, name, email, membershipType; methods: registerMember(), viewIssuedBooks(). ● **Librarian** — staffId, name, shift; methods: addBook(), removeBook(), approveIssue(). ● **IssueRecord** — recordId, issueDate, dueDate, returnDate; methods: calculateFine(), closeRecord().

b) Relationships Defined

- **Association:** Member creates IssueRecord; IssueRecord references Book; Librarian approves IssueRecord. ● **Aggregation:** Librarian manages Book — Books can exist in the catalogue independently of any single Librarian, so this is a whole-part (aggregation) relationship rather than composition.
- **Inheritance:** Member and Librarian can both inherit common attributes (id, name, contact) from an abstract base class “Person”, avoiding duplication.

c) Multiplicity

- Member (1) — IssueRecord (0..*): a member may have zero or many issue records.
- Book (1) — IssueRecord (0..*): a book may be referenced in many issue records over time, but each record references exactly one book.
- Librarian (1) — Book (0..*): one librarian manages many books; a book is catalogued by the library, not owned by one librarian.
- Librarian (1) — IssueRecord (0..*): a librarian approves many issue transactions.

d) Class Diagram

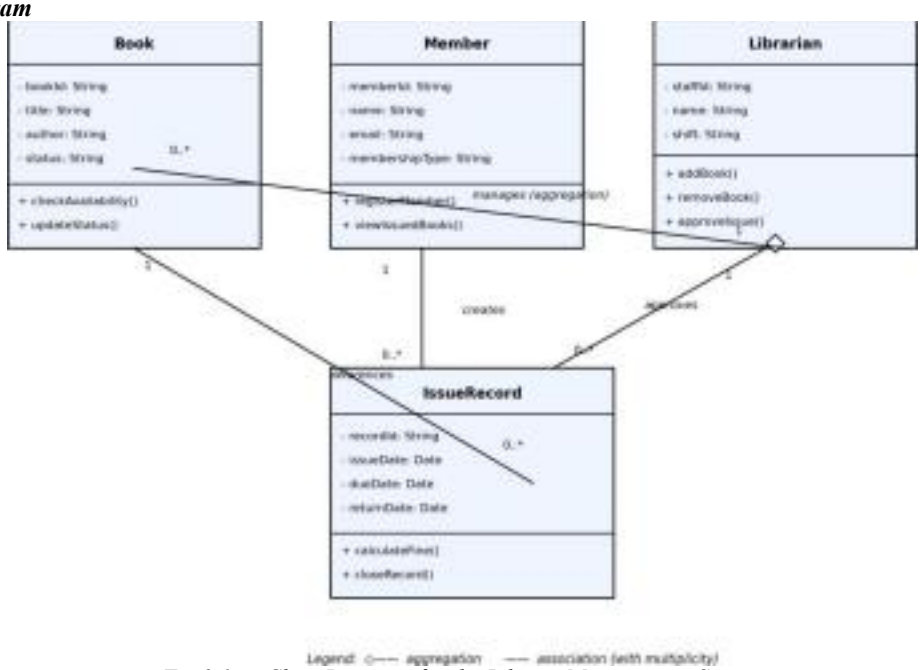

Fig 2.1 — Class Diagram for the Library Management System

3. Sequence Diagram Problem

In an **Online Food Ordering System**, a customer selects food items, places an order, makes payment, and receives order confirmation. The interaction flow is unclear.

Tasks:
a) Identify objects involved in the process.
b) Define message flow between objects.
c) Represent activation and lifelines.
d) Construct a Sequence Diagram showing complete interaction flow.

(10 Marks)

Answer

a) Objects Involved

- Customer — the actor initiating the order.
- OrderUI — the front-end screen the customer interacts with.

- OrderController — the back-end object that coordinates the order workflow.
- PaymentService — validates and processes payment transactions.
- Restaurant (Kitchen) — receives and prepares the confirmed order.

b) Message Flow Between Objects

- 1: Customer → OrderUI : selectFoodItems()
- 2: OrderUI → OrderController : placeOrder(items)
- 3: OrderController → Restaurant : checkItemAvailability()
- 4: Restaurant --> OrderController : availabilityConfirmed() (return message)
- 5: OrderController → PaymentService : initiatePayment(amount)
- 6: PaymentService (self-call) : validate & process
- 7: PaymentService --> OrderController : paymentSuccess() (return message)
- 8: OrderController → Restaurant : forwardOrder()
- 9: OrderController --> OrderUI : orderConfirmed() (return message)
- 10: OrderUI --> Customer : showConfirmation() (return message)

c) Activation and Lifelines

- Each object has a dashed vertical lifeline running down the diagram, representing its existence over time. ● A narrow activation bar is shown on each lifeline for the duration that object is actively processing a request (e.g. OrderController remains active from message 2 until message 9 completes).
- Solid arrows represent synchronous calls; dashed arrows represent return messages.

d) Sequence Diagram

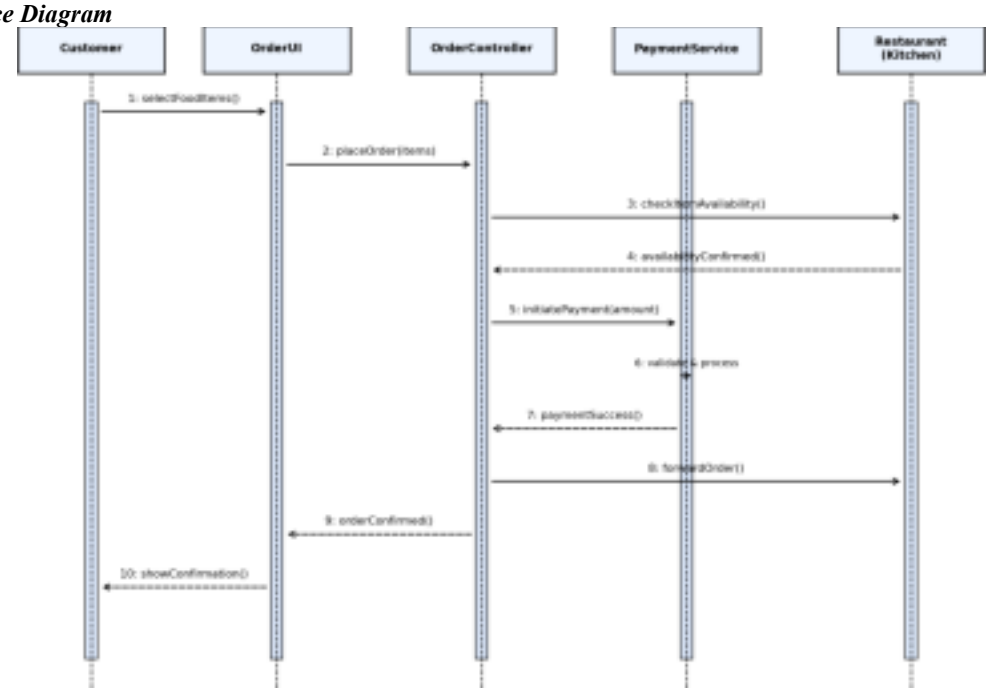

Fig 3.1 — Sequence Diagram for the Online Food Ordering System

4. Activity Diagram Problem

A **Student Admission System** includes application submission, document verification, fee payment, and enrollment confirmation. The workflow lacks proper decision-making steps.

Tasks:
a) Identify all activities in the workflow.
b) Add decision nodes for approval/rejection.
c) Include parallel processing where applicable.
d) Design an optimized Activity Diagram.

(10 Marks)

Answer

a) Activities Identified

- Submit Application
- Verify Documents
- Check Eligibility
- Reject Application & Notify Applicant
- Fee Payment
- Enrollment Confirmation
- Send Welcome Notification

b) Decision Nodes for Approval / Rejection

- After document verification and eligibility checking are complete, a decision node evaluates 'Documents Valid?'.
- [No] branch → Reject Application & Notify Applicant → workflow ends.
- [Yes] branch → proceeds to Fee Payment → Enrollment Confirmation.

c) Parallel Processing

- A fork node splits the flow after 'Submit Application' into two concurrent activities: 'Verify Documents' and 'Check Eligibility', since these checks are independent and can run simultaneously to reduce processing time.
- A join node synchronizes both branches before the decision node is evaluated, ensuring both checks have completed.

d) Activity Diagram

Fig 4.1 — Activity Diagram for the Student Admission System